

Clinical Appeals Review Copilot

End-to-End Technical Documentation

Multi-Channel Intake • Deterministic Rules Engine • Policy-Grounded RAG Evaluation • Human-in-the-Loop Review

Internal project — MVP1

September 2026

1. Executive Summary

The Appeals & Grievances Review Copilot is an agentic AI system that evaluates member appeals of denied medical claims against the original denial, the member's benefit and eligibility records, treatment history, and the health plan's clinical policy bulletins — producing a recommended outcome for a human reviewer to confirm. It accepts appeals through two channels, email and direct upload, and processes both through the same nine-stage evaluation path.

Pillar	Key Capabilities
Deterministic-First Evaluation	7 of 8 decision checks run as auditable business rules against structured reference data before any AI reasoning is invoked
Multi-Channel Intake	Email connector (Microsoft Graph) and direct upload, normalized into one case record regardless of source format
Policy-Grounded Recommendations	Every clinical judgment is retrieved from and cited against the plan's own policy bulletins — no uncited claims
Human-in-the-Loop	No recommendation becomes a final decision without appeals-staff confirmation

Note: Ten distinct case outcomes — one approval path, five denial reasons, a documentation-gap hold, an identity hold, a duplicate merge, and an escalation path — are handled by the same evaluation pipeline, described in full in Section 9.

2. Technology Stack

Layer	Technology	Notes
Frontend	React + Vite, Tailwind	Reviewer dashboard and upload form
API Framework	FastAPI	Async-first; intake and case endpoints
Orchestration	LangGraph	State machine; the case record is shared state across all nodes
LLM — extraction / summarization	Azure OpenAI GPT-4o-mini	Intake Agent, Summarization Agent
LLM — evaluation	Azure OpenAI GPT-4o	Evaluation Agent — the highest-stakes reasoning step
OCR	Azure AI Vision — Read API	Document text extraction, both intake channels
PHI/PII Guardrail	Microsoft Presidio	Tokenizes identifiers before any LLM call
Vector Database	ChromaDB	Policy bulletin semantic retrieval only — not reference data
Relational Database	PostgreSQL	Case records, structured reference data, audit trail

Layer	Technology	Notes
Email Ingestion	Microsoft Graph API	Monitored mailbox connector
Deployment	Docker Compose, single VM (MVP1)	Migrates to Azure AKS / APIM / Cosmos DB in production
Tracing	LangSmith	Per-agent trace for debugging and audit

3. Project Structure

```

appeals-review-mvp/
├── app/
│   ├── api/
│   │   ├── intake.py          # POST /intake/email/poll, /intake/upload
│   │   ├── cases.py           # GET /cases, GET /cases/{id}
│   │   ├── review.py          # POST /cases/{id}/review
│   │   └── chat.py            # POST /cases/{id}/chat
│   ├── core/
│   │   ├── graph.py           # Compiled LangGraph state machine
│   │   ├── nodes.py           # intake_agent, summarization_agent, evaluation_agent
│   │   └── decision_engine.py # Checks 1-7, deterministic rule chain
│   ├── security/
│   │   ├── redaction.py       # PHI/PII tokenization (Presidio)
│   │   └── audit.py           # Audit trail writer
│   ├── services/
│   │   ├── ocr_service.py
│   │   ├── lookup_service.py  # denial / eligibility / utilization reads
│   │   ├── policy_retrieval_service.py # ChromaDB semantic search
│   │   └── email_connector.py
│   ├── models.py              # Case, Denial, Benefits, Utilization schemas
│   └── main.py
├── reference_data/
│   ├── denials.json
│   ├── eligibility_benefits.json
│   ├── utilization_history.json
│   └── policy_bulletins/
├── test_data/
│   ├── emails/
│   └── medical_history/
├── docker-compose.yml
└── pyproject.toml

```

4. System Architecture

4.1 High-Level Case Flow

Step	Component	Description
1	Intake Connectors	Email connector (Graph API) or upload endpoint creates a Case record — channel and timestamp recorded, nothing interpreted yet
2	Ingestion Service	OCR / text extraction normalizes every document into one working text pool
3	PHI/PII Guardrail	Presidio tokenizes identifiers before any LLM call; non-essential identifiers are stripped outright
4	Intake Agent	Extracts structured fields from tokenized text; resolves identity via member_id or a name+DOB fallback match
5	Lookup Service	Deterministic keyed reads: denial record, eligibility & benefits, utilization history
6	Summarization Agent	Condenses the case into a short reviewer-facing summary

Step	Component	Description
7	Rules Engine + Evaluation Agent	Runs the eight-check decision sequence (Section 5) — only check 8 invokes an LLM
8	Reviewer Dashboard	Surfaces case, recommendation, and citations; records the human decision
9	Case Finalization	Writes the final record and the complete audit trail

4.2 LangGraph State Machine

Node	Trigger	Output
intake_agent	Entry point, after ingestion + redaction	Structured case fields; identity resolved or flagged missing
lookup_records	After intake_agent, if identity confirmed	denial, benefits, and utilization objects attached to the case
rules_engine	After lookup_records	Either a conclusive outcome (checks 2–7) or a pass-through to evaluation_agent
summarization_agent	In parallel with rules_engine	clinical_summary field populated
evaluation_agent	Only if rules_engine passes all of checks 2–7	decision, rationale, per-criterion findings, citations (check 8)
finalize	Always last	Case status updated, audit trail entry written, reviewer queue notified

5. Evaluation Pipeline

5.1 Decision Sequence Overview

Stage 7 of the case flow is not a single decision — it is an ordered sequence of eight checks, cheapest and most certain first, stopping at the first check that produces a conclusive answer. Only the last check requires an AI model.

Check	Type	Determines	Outcome if triggered
1	Deterministic	Can the member be identified?	Scenario 8 — Cannot process
2	Deterministic	Does an open case already exist for this reference?	Scenario 9 — Duplicate, merge
3	Deterministic	Was a prior internal appeal on this denial already upheld?	Scenario 10 — Escalate to external review
4	Deterministic	Was the appeal filed within the appeal window?	Scenario 6 — Deny, filed after deadline
5	Deterministic	Was coverage active on the date of service?	Scenario 5 — Deny, not eligible on date of service
6	Deterministic	Is the requested service a covered benefit?	Scenario 3 — Deny, not a covered benefit
7	Deterministic	Does the request stay within the annual benefit allowance?	Scenario 4 — Deny (no override), or Scenario 7 — Needs info (partial override)
8	Agentic (AI)	Does the case meet every criterion in the applicable clinical policy?	Scenario 1 — Approve, Scenario 2 — Deny, or Scenario 7 — Needs info

5.2 Identity Resolution

Check 1 first attempts a direct match on `member_id`. When `member_id` is absent but the source document contains a full name and date of birth, a fallback lookup attempts a match on those two fields together. A first

name alone, or a name with no date of birth, is never sufficient — that case is held (Scenario 8) rather than risking a match against the wrong member.

5.3 Policy Retrieval (RAG)

Policy bulletins are chunked and embedded into ChromaDB ahead of time. Check 8 retrieves relevant passages using the denial's cited_policy_id combined with a semantic query built from the case's clinical evidence, then evaluates the case against each numbered criterion in the retrieved bulletin individually.

5.4 Citation Guardrail

Every claim in the Evaluation Agent's rationale must trace to a specific retrieved passage. A claim with no supporting citation is rejected and the agent is required to retry — this is what keeps a denial or approval traceable to an actual policy sentence rather than the model's general sense of what sounds reasonable.

6. Human-in-the-Loop Review Workflow

Step	Actor	Description
1. Case drafted	System	Evaluation Agent produces a decision, rationale, and citations; case status moves to Drafted
2. Queue	Reviewer Dashboard	Case appears in the reviewer queue, sorted by status
3. Review	Appeals staff	Reads the summary, recommendation, source documents, and reference data
4. Decision	Appeals staff	Approve, edit, or reject the recommendation
5. Record	System	Decision, reviewer name, and date written to the audit trail; status moves to Reviewed
6. Close	System	Final record stored; status moves to Closed

7. Security & Privacy Architecture

Layer	ID	Module	What it does
Capture Validation	L1	intake connectors	Rejects malformed email or file payloads before a case is opened
PII/PHI Tokenization	L2	security/redaction.py	Presidio masks and tokenizes identifiers before any LLM call
Data Labeling	L3	agent prompts	Retrieved policy and document text is labeled as data, not instructions, in every agent prompt
Citation Enforcement	L4	evaluation_agent	Every clinical claim must cite a retrieved policy passage
Audit Logging	L5	core/audit.py	Every status transition and reviewer action is recorded, immutably
Access Control	L6	reviewer dashboard	Authenticated access only — not yet implemented in MVP1

Warning: L6 (Access Control) is not implemented in MVP1 — the reviewer dashboard currently has no authentication. This is acceptable for a local prototype running on synthetic data only, and must be addressed before the system is exposed to real member data.

8. Data Model & Reference Data

8.1 Case Schema

```

Case {
  case_id, channel, status
  member: { member_id, name, date_of_birth }
  provider: { name, npi, organization }
  request: { procedure, cpt_code, denial_reference }
  diagnosis_codes: []
  documents: [{ filename, source, storage_path, ocr_text }]
  clinical_summary
  denial: { denial_reference, stated_reason, cited_policy_id, appeal_deadline }
  benefits: { plan_status, network_status, annual_limit, units_used_ytd }
  utilization: [{ date, type, service, outcome }]
  recommendation: { decision, rationale, cited_guidelines: [] }
  audit_trail: [{ timestamp, actor, action }]
}

```

8.2 Reference Data Stores

Store	Format	Keyed by	Contents
Denial Records	Structured (Postgres / JSON)	denial_reference	Original denial reason, cited policy ID, reviewer notes, appeal deadline
Eligibility & Benefits	Structured (Postgres / JSON)	member_id	Plan status, network status, benefit caps, units used
Utilization History	Structured (Postgres / JSON)	member_id	Prior claims, prior authorizations, prior appeals
Policy Bulletins	Vector store (ChromaDB)	policy_id + semantic	Payer medical-necessity criteria documents, chunked and embedded

Note: Only the policy bulletins belong in the vector store. The other three stores need exact-match lookups by ID, which a similarity search handles unreliably — they stay in structured tables.

9. Outcome Scenarios Reference

#	Scenario	Outcome	Trigger
1	Missing evidence supplied	Approved	The exact gap the denial cited turns out to be documented after all
2	Clinical requirements not met	Denied	Thresholds genuinely aren't met, and the appeal adds no new evidence
3	Not a covered benefit	Denied	Plan excludes the service unless a specific condition is documented — it isn't
4	Annual limit exceeded, no override	Denied	Exceeds a hard cap with none of the required override evidence submitted
5	Not eligible on date of service	Denied	Coverage had lapsed on the day the service was provided
6	Filed after the deadline	Denied	Appeal arrived after the filing window closed, with no reason given
7	Specific documentation gap	Needs info	Most requirements met; one identifiable item is missing
8	Member can't be identified	On hold	Not enough information to safely match a member record
9	Same appeal received twice	Merged	Duplicate submission of an appeal already in progress
10	Prior internal appeal already upheld	Escalated	Routed to external review rather than a repeat internal look

10. API Design

Endpoint	Method	Purpose
/intake/email/poll	POST	Pulls new mailbox items; creates a Case per email

Endpoint	Method	Purpose
/intake/upload	POST	Multipart file upload; creates a Case with channel: upload
/cases	GET	List cases, filterable by status — backs the reviewer queue
/cases/{case_id}	GET	Full case record: documents, summary, recommendation, audit trail
/cases/{case_id}/review	POST	Reviewer action — approve / edit / reject; writes an audit event
/cases/{case_id}/chat	POST	Case-scoped Q&A grounded in that case's documents and summary
/cases/{case_id}/audit	GET	Audit trail only, for a lighter-weight history view

11. Setup & Deployment

11.1 Prerequisites

- Python 3.12+
- Docker and Docker Compose
- Azure OpenAI access (GPT-4o and GPT-4o-mini)
- Microsoft Graph API app registration (mailbox read access)
- A small starting set of clinical policy bulletins to seed ChromaDB

11.2 Local Development

```
# 1. Clone and set up the environment
git clone <repo-url> && cd appeals-review-mvp
uv venv && source .venv/bin/activate
uv pip install -e '.[dev]'

# 2. Configure environment
cp .env.example .env # Azure OpenAI key, Graph API credentials, DB URL

# 3. Start infrastructure
docker compose up -d # PostgreSQL, ChromaDB

# 4. Load reference data
make load-reference-data # denials, eligibility, utilization, policy bulletins

# 5. Start the API
uvicorn app.main:app --reload

# API: http://localhost:8000 | Docs: http://localhost:8000/docs
```

11.3 Implementation Phases

Phase	Focus	Validates
1	Rules engine (checks 1–7), no AI at all	Scenarios 3, 4, 5, 6, 8, 9, 10
2	Intake Agent	Scenario 8 from real unstructured input, not pre-filled test data
3	Evaluation Agent + starting policy library	Scenarios 1, 2, 7
4	Summarization Agent + reviewer dashboard	End-to-end reviewer experience
5	Full regression across all ten scenarios	All scenarios, including the negative ones

12. Testing & Evaluation

12.1 Intake Test Data

Six synthetic files — three .eml emails and three medical-history PDFs — covering a clean structured extraction case, a missing-identity edge case, and a multi-attachment / multi-page case. A ground-truth manifest (expected_extraction.json) gives the correct structured output for each, for automated regression testing of the Intake Agent.

12.2 Evaluation Reference Data

Denial, eligibility, and utilization records plus three policy bulletins, keyed to match the intake test data exactly, with an evaluation manifest describing which lookups should succeed and what the Evaluation Agent's reasoning should conclude for each case.

12.3 Coverage Gaps

The current test data only backs Scenarios 1, 7, and 8 with real example files. Full coverage of checks 2 through 6 and the harder version of check 7 (Scenario 4) still needs:

- Denial, eligibility, and utilization records for the five denial examples (Yeung, Anand, Okafor, Brooks, Reyes)
- A duplicate-submission test pair — two near-identical emails referencing the same case number
- An already-upheld appeal record for the escalation check, including the date and outcome of the first internal review

13. Glossary

Term	Definition
CRT-D	Cardiac resynchronization therapy defibrillator — an implanted heart device
GDMT	Guideline-directed medical therapy — the standard medication regimen for heart failure
ICD-10	Standard diagnosis coding system
CPT	Standard procedure coding system used for billing
Prior Authorization	Payer approval required before a service is covered
NYHA Class	New York Heart Association heart-failure severity classification
PHI / PII	Protected health information / personally identifiable information
RAG	Retrieval-Augmented Generation — retrieving relevant text before an LLM answers; used here for policy bulletins only
LangGraph	Graph-based orchestration framework for stateful, multi-agent workflows
Deterministic Check	A rule or lookup with no AI involvement — the same input always gives the same output
Guardrail	A rule that constrains an AI agent's output, such as a citation requirement or a no-fabrication rule
Escalation	Routing a case to external review after one internal appeal has already been upheld
Reference Data	Structured records (denial, eligibility, utilization) used to evaluate a case, distinct from the case's own submitted documents